

INSTITUTO FEDERAL
Brasília



Lista de Exercícios 3

Programação Orientada a Objetos em Python

Orientações gerais

Em todos os exercícios:

1. Utilize classes e objetos para organizar o programa.
2. Implemente construtores com o método `__init__()`.
3. Utilize nomes de classes iniciando com letra maiúscula.
4. Utilize nomes de atributos e métodos em letras minúsculas.
5. Comente as partes mais importantes do código.
6. Teste o funcionamento das classes no programa principal.
7. Nos exercícios que solicitarem menu, mantenha o programa em execução até que o usuário escolha a opção de sair.

Nível básico

1. Cadastro de pessoa

Crie uma classe chamada `Pessoa` com os atributos `nome`, `idade` e `cidade`. Implemente um método chamado `apresentar()` que exiba uma mensagem com todos os dados da pessoa.

No programa principal, crie dois objetos da classe `Pessoa` e execute o método `apresentar()`.

Conceitos trabalhados: classe, objeto, atributos, construtor e método.

2. Cálculo da área de um retângulo

Crie uma classe chamada `Retangulo` com os atributos `base` e `altura`. Implemente os métodos:

- `calcular_area()`
- `calcular_perimetro()`
- `exibir_dados()`

O programa deve solicitar os valores ao usuário e mostrar a área e o perímetro calculados.

Conceitos trabalhados: atributos, métodos e retorno de valores.

3. Controle de temperatura

Crie uma classe chamada `Temperatura` que armazene uma temperatura em graus Celsius.

Implemente métodos para:

- converter Celsius para Fahrenheit;
- converter Celsius para Kelvin;
- exibir os três valores.

Utilize as fórmulas:

Fahrenheit = Celsius × 1,8 + 32
Kelvin = Celsius + 273,15

Conceitos trabalhados: classe, métodos e cálculos dentro de objetos.

4. Produto com desconto

Crie uma classe chamada `Produto` com os atributos `nome`, `preco` e `percentual_desconto`.

Implemente os métodos:

- `calcular_desconto()`
- `calcular_preco_final()`
- `exibir_resumo()`

O programa deve mostrar o preço original, o valor do desconto e o preço final.

Conceitos trabalhados: estado do objeto e comportamento.

5. Aluno e situação acadêmica

Crie uma classe chamada `Aluno` com os atributos `nome`, `nota1` e `nota2`.

Implemente métodos para:

- calcular a média;
- verificar a situação do aluno;
- exibir o resultado.

Considere:

- média maior ou igual a 6: aprovado;
- média maior ou igual a 4 e menor que 6: recuperação;
- média menor que 4: reprovado.

Conceitos trabalhados: métodos, decisões e encapsulamento de regras.

6. Conta de energia elétrica

Crie uma classe chamada `ContaEnergia` com os atributos `nome_cliente`, `consumo_kwh` e `valor_kwh`.

Implemente um método para calcular o valor da conta e outro para exibir os dados do cliente e o total a pagar.

Não permita que o consumo seja negativo.

Conceitos trabalhados: validação simples e métodos.

7. Agenda de contatos

Crie uma classe chamada `Contato` com os atributos `nome`, `telefone` e `email`.

Crie também uma classe chamada `Agenda`, responsável por armazenar uma lista de contatos.

A agenda deve permitir:

1. adicionar contato;

2. listar contatos;
3. buscar contato pelo nome;
4. sair.

Conceitos trabalhados: objetos armazenados em listas e associação entre classes.

8. Controle de lâmpada

Crie uma classe chamada `Lampada` com os atributos `cor`, `potencia` e `ligada`.

Implemente os métodos:

- `ligar()`
- `desligar()`
- `mostrar_estado()`

O programa deve impedir que a lâmpada seja ligada novamente caso já esteja ligada.

Conceitos trabalhados: alteração do estado de objetos.

9. Funcionário com reajuste salarial

Crie uma classe chamada `Funcionario` com os atributos `nome`, `cargo` e `salario`.

Implemente métodos para:

- aplicar um reajuste percentual;
- exibir os dados do funcionário;
- consultar o salário atual.

O salário deve ser privado e somente poderá ser alterado pelos métodos da classe.

Conceitos trabalhados: encapsulamento, getters e métodos de alteração.

10. Livro com controle de disponibilidade

Crie uma classe chamada `Livro` com os atributos `titulo`, `autor` e `disponivel`.

Implemente os métodos:

- `emprestar()`
- `devolver()`
- `exibir_status()`

O programa deve impedir o empréstimo de um livro que já esteja emprestado.

Conceitos trabalhados: controle de estado e regras dentro da classe.

Nível médio

11. Sistema de pedidos de restaurante

Crie as classes `Produto`, `ItemPedido` e `Pedido`.

A classe `Produto` deve armazenar nome e preço.

A classe `ItemPedido` deve armazenar um produto e a quantidade solicitada.

A classe `Pedido` deve manter uma lista de itens e permitir:

1. adicionar item;
2. remover item;
3. listar itens;
4. calcular o total;
5. finalizar o pedido.

Após a finalização, nenhum novo item poderá ser adicionado.

Conceitos trabalhados: associação, composição e regras de negócio.

12. Estacionamento

Crie as classes `Veiculo` e `Estacionamento`.

Cada veículo deve possuir placa, modelo e horário de entrada.

O estacionamento deve permitir:

1. registrar entrada;
2. registrar saída;
3. listar veículos estacionados;
4. calcular o valor a pagar.

Considere o valor fictício de R$ 5,00 por hora ou fração.

Conceitos trabalhados: associação, listas de objetos e manipulação de datas.

13. Sistema de animais

Crie uma classe base chamada `Animal`, com os atributos `nome` e `idade`, e um método chamado `emitir_som()`.

Crie as subclasses:

- `Cachorro`
- `Gato`
- `Vaca`

Cada subclasse deverá sobrescrever o método `emitir_som()`.

Armazene diferentes animais em uma única lista e execute o método `emitir_som()` para todos.

Conceitos trabalhados: herança, sobrescrita e polimorfismo.

14. Pagamento de funcionários

Crie uma classe base chamada `Funcionario`, com nome e salário-base.

Crie as subclasses:

- `FuncionarioHorista`
- `FuncionarioComissionado`
- `FuncionarioTemporario`

Cada classe deve implementar seu próprio método `calcular_pagamento()`.

Regras:

- horista: horas trabalhadas multiplicadas pelo valor da hora;
- comissionado: salário-base somado à comissão;
- temporário: quantidade de dias trabalhados multiplicada pelo valor diário.

Conceitos trabalhados: herança e polimorfismo.

15. Controle de estoque

Crie as classes `Produto` e `Estoque`.

Cada produto deve possuir código, nome, preço e quantidade.

O estoque deve permitir:

1. cadastrar produto;
2. adicionar unidades;
3. retirar unidades;
4. consultar produto pelo código;
5. listar produtos com estoque abaixo de cinco unidades;
6. calcular o valor total armazenado.

Utilize exceções para impedir retiradas maiores que a quantidade disponível.

Conceitos trabalhados: encapsulamento, exceções e agregação.

16. Sistema de reservas de hotel

Crie as classes `Hospede`, `Quarto` e `Reserva`.

Cada quarto deve possuir número, tipo, valor da diária e disponibilidade.

A reserva deve armazenar o hóspede, o quarto e a quantidade de diárias.

O sistema deve permitir:

1. cadastrar hóspedes;
2. listar quartos disponíveis;
3. realizar reserva;
4. cancelar reserva;
5. calcular o valor da hospedagem.

Conceitos trabalhados: associação entre objetos e regras de disponibilidade.

17. Catálogo de filmes e séries

Crie uma classe base chamada `Conteudo`, com título, gênero e duração.

Crie as subclasses:

- `Filme`, com o atributo `diretor`;
- `Serie`, com os atributos `quantidade_temporadas` e `quantidade_episodios`.

Implemente o método `exibir_detalhes()` de forma diferente em cada subclasse.

Crie um catálogo capaz de armazenar filmes e séries na mesma lista.

Conceitos trabalhados: herança, polimorfismo e coleções de objetos.

18. Sistema de notificações

Crie uma classe abstrata chamada `Notificador`, contendo o método abstrato `enviar(mensagem)`.

Crie as subclasses:

- `NotificadorEmail`
- `NotificadorSMS`
- `NotificadorAplicativo`

Cada uma deve simular o envio da mensagem de uma forma diferente.

Crie uma função que receba qualquer objeto do tipo `Notificador` e execute o envio.

Conceitos trabalhados: abstração, polimorfismo e dependência.

19. Carrinho com diferentes formas de pagamento

Crie as classes `Carrinho`, `Produto` e uma classe abstrata chamada `FormaPagamento`.

Implemente as formas de pagamento:

- `PagamentoDinheiro`
- `PagamentoCartao`
- `PagamentoPix`

Cada forma de pagamento deve implementar o método `pagar(valor)`.

Regras:

- dinheiro: aplicar desconto de 5%;
- cartão: acrescentar taxa de 2%;
- Pix: aplicar desconto de 3%.

Conceitos trabalhados: abstração, estratégia e polimorfismo.

20. Sistema de chamados de suporte

Crie as classes `Usuario`, `Tecnico` e `Chamado`.

Um chamado deve possuir número, descrição, prioridade, status, usuário solicitante e técnico responsável.

O sistema deve permitir:

1. abrir chamado;
2. atribuir técnico;
3. alterar prioridade;
4. registrar atendimento;
5. encerrar chamado;
6. listar chamados por status.

Mantenha um histórico das alterações realizadas em cada chamado.

Conceitos trabalhados: associação, composição, encapsulamento e histórico de objetos.

Nível avançado. Questões-desafio

21. Sistema de locação de veículos

Desenvolva um sistema completo de locação de veículos.

Crie uma classe abstrata `Veiculo` e as subclasses:

- `Carro`
- `Moto`
- `Van`

Cada tipo deve possuir uma regra diferente para calcular o valor da diária.

Crie também as classes:

- `Cliente`
- `Locacao`
- `Agencia`

O sistema deve permitir:

1. cadastrar veículos;
2. cadastrar clientes;
3. realizar locações;
4. devolver veículos;
5. calcular multas por atraso;
6. listar veículos disponíveis;
7. consultar o histórico de locações de um cliente.

Crie exceções personalizadas para veículo indisponível, cliente inexistente e período inválido.

22. Plataforma de cursos com avaliação

Desenvolva uma plataforma de cursos com as classes:

- `Usuario`
- `Aluno`
- `Professor`
- `Curso`
- `Aula`
- `Matricula`
- `Avaliacao`

O professor poderá criar cursos e aulas.

O aluno poderá se matricular, concluir aulas e realizar avaliações.

O sistema deverá:

1. calcular o progresso percentual do aluno;
2. impedir avaliações antes da conclusão mínima de 70% do curso;
3. calcular a média das avaliações;
4. emitir certificado para alunos aprovados;
5. manter o histórico de matrículas.

Utilize classes abstratas, herança, composição, polimorfismo e exceções personalizadas.

23. Sistema de entregas

Crie uma classe abstrata chamada `Entrega`, com os métodos abstratos:

- `calcular_frete()`
- `calcular_prazo()`

- `rastrear()`

Crie as subclasses:

- `EntregaNormal`
- `EntregaExpressa`
- `EntregaRetiradaLocal`

Considere peso, distância, região e tipo de entrega no cálculo do valor.

Crie também as classes `Cliente`, `Endereco`, `Pedido` e `Transportadora`.

O sistema deve acompanhar os estados:

Pedido criado
Pagamento confirmado
Em separação
Enviado
Em transporte
Entregue

Impeça alterações inválidas de estado.

24. Sistema de controle hospitalar

Desenvolva um sistema com as classes:

- `Pessoa`
- `Paciente`
- `Medico`
- `Enfermeiro`
- `Consulta`
- `Prontuario`
- `Prescricao`

Um prontuário deve pertencer exclusivamente a um paciente.

O sistema deve permitir:

1. cadastrar pacientes e profissionais;
2. agendar consultas;
3. registrar diagnósticos;
4. adicionar prescrições;
5. consultar o histórico do paciente;
6. impedir conflitos de horário para o mesmo médico;

7. controlar o acesso ao prontuário conforme o tipo de profissional.

Crie exceções personalizadas para conflitos de agenda e acesso não autorizado.

25. Sistema de investimento financeiro

Crie uma classe abstrata chamada `Investimento`.

Implemente as subclasses:

- `Poupanca`
- `CDB`
- `FundoInvestimento`
- `Acao`

Cada investimento deverá implementar:

- `calcular_rendimento()`
- `calcular_imposto()`
- `exibir_risco()`

Crie as classes `Investidor` e `CarteiraInvestimentos`.

O sistema deverá:

1. adicionar investimentos à carteira;
2. calcular o patrimônio total;
3. calcular os rendimentos;
4. calcular os impostos;
5. mostrar a distribuição percentual dos investimentos;
6. apresentar os investimentos por nível de risco.

Utilize polimorfismo para calcular os resultados.

26. Sistema de transporte urbano

Crie uma classe abstrata chamada `Transporte`, com os métodos:

- `calcular_tarifa()`
- `calcular_tempo_estimado()`
- `exibir_rota()`

Crie as subclasses:

- `Onibus`
- `Metro`
- `BicicletaCompartilhada`
- `TransporteAplicativo`

Crie também as classes `Passageiro`, `Viagem`, `Rota` e `CartaoTransporte`.

O sistema deverá:

1. calcular tarifas diferentes;
2. permitir integração tarifária;
3. verificar saldo do cartão;
4. registrar o histórico de viagens;
5. calcular a duração estimada;
6. sugerir o transporte de menor custo.

27. Sistema de gerenciamento de projetos

Crie as classes:

- `Usuario`
- `Gerente`
- `Desenvolvedor`
- `Projeto`
- `Tarefa`
- `Comentario`
- `Equipe`

Uma tarefa deve possuir título, descrição, responsável, prazo, prioridade e status.

Implemente os estados:

Pendente
Em andamento
Em revisão
Concluída
Cancelada

O sistema deve:

1. cadastrar projetos;
2. formar equipes;
3. criar tarefas;
4. atribuir responsáveis;
5. registrar comentários;

6. impedir transições inválidas de status;
7. identificar tarefas atrasadas;
8. calcular o percentual de conclusão do projeto;
9. gerar um relatório do projeto.

Utilize encapsulamento, composição, herança e exceções personalizadas.

28. Jogo de batalha por turnos

Desenvolva um jogo utilizando as classes:

- `Personagem`
- `Guerreiro`
- `Mago`
- `Arqueiro`
- `Habilidade`
- `Item`
- `Batalha`

A classe `Personagem` deverá ser abstrata.

Cada personagem deve possuir vida, energia, nível, defesa e habilidades.

O sistema deverá:

1. controlar os turnos;
2. calcular ataques e defesas;
3. utilizar habilidades especiais;
4. consumir energia;
5. permitir o uso de itens;
6. impedir ações de personagens derrotados;
7. encerrar a batalha quando restar apenas um vencedor;
8. registrar todo o histórico da batalha.

Utilize polimorfismo nos ataques e nas habilidades.

29. Sistema de comércio eletrônico com persistência

Desenvolva um sistema de comércio eletrônico com as classes:

- `Cliente`
- `Endereco`
- `Produto`

- `Carrinho`
- `ItemCarrinho`
- `Pedido`
- `Pagamento`
- `Cupom`

O sistema deverá:

1. cadastrar clientes e produtos;
2. adicionar e remover produtos do carrinho;
3. controlar estoque;
4. aplicar cupons;
5. calcular frete;
6. realizar pagamento;
7. gerar pedido;
8. atualizar o status do pedido;
9. salvar os dados em arquivos JSON;
10. carregar os dados automaticamente ao iniciar.

Implemente diferentes formas de pagamento utilizando uma classe abstrata.

Garanta que os arquivos não sejam corrompidos caso ocorra um erro durante a gravação.

30. Desafio final. Sistema integrado para gestão acadêmica

Desenvolva um sistema completo de gestão acadêmica.

O sistema deve possuir, no mínimo, as classes:

- `Pessoa`
- `Aluno`
- `Professor`
- `Coordenador`
- `Curso`
- `Disciplina`
- `Turma`
- `Matricula`
- `Avaliacao`
- `Frequencia`
- `HistoricoEscolar`

Requisitos obrigatórios:

1. Uma pessoa não pode ser instanciada diretamente.
2. Aluno, professor e coordenador devem herdar de `Pessoa`.
3. Um curso deve possuir várias disciplinas.
4. Uma turma deve relacionar professor, disciplina e alunos.
5. O aluno deve possuir matrícula única.
6. O sistema deve registrar notas e frequências.
7. A média final deve ser calculada automaticamente.
8. O aluno deverá ser reprovado por falta quando possuir frequência inferior a 75%.
9. O professor somente poderá registrar notas em suas próprias turmas.
10. O coordenador poderá criar cursos, disciplinas e turmas.
11. O histórico escolar deverá mostrar disciplinas cursadas, notas, frequências e situações.
12. O sistema deverá utilizar exceções personalizadas.
13. Os dados deverão ser armazenados em arquivos JSON.
14. O programa deverá possuir um menu interativo.
15. O código deverá estar dividido em módulos ou arquivos.
16. O sistema deverá possuir tratamento para arquivos inexistentes ou corrompidos.
17. Deverá ser demonstrado o uso de encapsulamento, herança, abstração, polimorfismo, associação, agregação, composição e dependência.

Como requisito adicional, produza um pequeno diagrama textual explicando o relacionamento entre as classes antes de iniciar a implementação.